Data Structure 4

Paul Blondel

CNAM

September 1st, 2018

'I will, in fact, claim that the difference between a bad programmer and a good one is whether he considers his code or his data structures more important. Bad programmers worry about the code. Good programmers worry about data structures and their relationships.'

Linus Torvalds

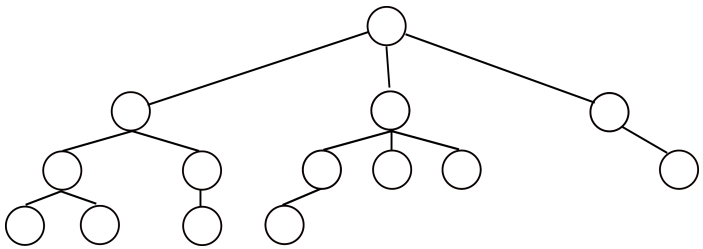
1. Trees
   - Definition
   - Traversing and reading a tree
2. Binary trees
   - Properties
   - Complete and almost complete binary trees
   - Binary heaps
3. Binary search trees (BST)
   - Properties
4. AVL trees
   - Properties
   - Simple rotation and double rotations
5. B-trees
   - Properties
6. Summary

What are trees?

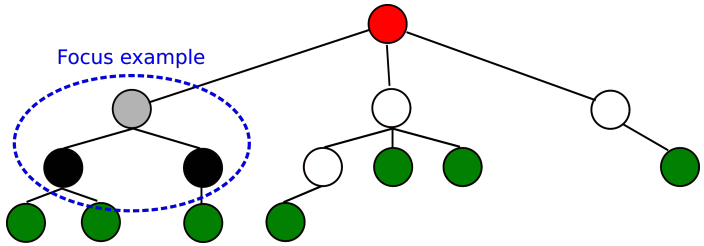
A tree is a data structure...

- ...with a root element (**root node**)
- ...followed by other elements (or **nodes**):
  - organized in a **family of subsets**
  - family subsets organized in **a tree structure**

Example of tree:



Not all nodes have the same name...

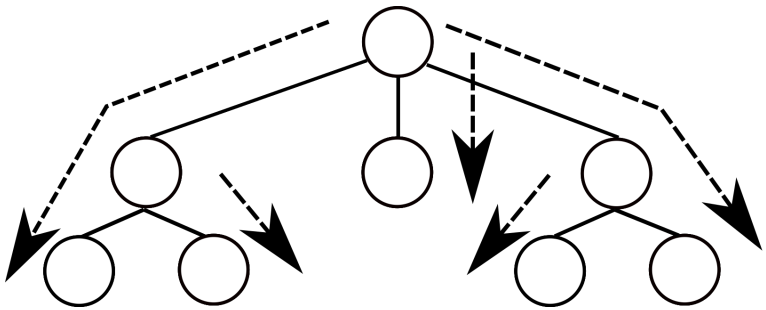


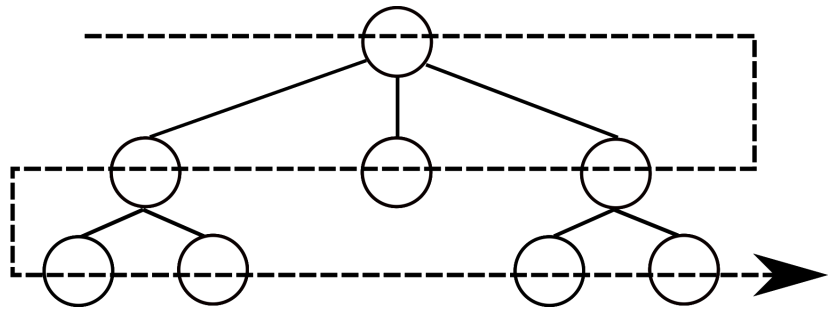
- In *red*: the **root node** (*super parent*)
- In *green*: the **node leaves** (like a leaf of a tree...)
- Focus example: a parent node (*grey*) with two children nodes (*black*)

Trees can be traversed in ***depth-first* or *breadth-first***[3] order:

- *depth-first order*: **traverse to deepest levels** before other children
- *breadth-first order*: all nodes are traversed **level after level**

Depth-first traversal (left side) and *breadth-first* traversal (right side):



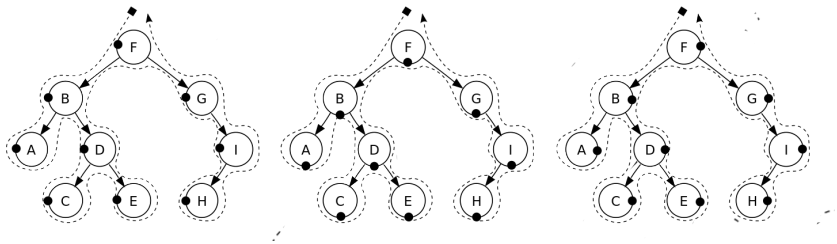


[3] We won't talk much about *breadth-first* order traversal, as it is less used.

Three ways of *reading node contents* with *depth-first* traversal[3]:

- *Pre-order*: directly read the **visited node content**
- *In-order*: traverse **left subtree and read visited node content**
- *Post-order*: traverse **all subtrees and read visited node content**

Examples:

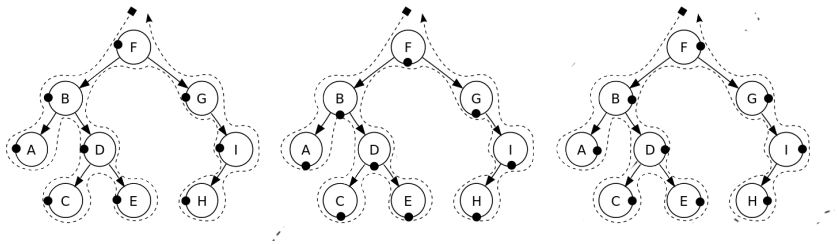


[3]By default, node contents are read in *pre-order* mode

Three ways of *reading node contents* with *depth-first* traversal[3]:

- *Pre-order*: directly read the **visited node content**
- *In-order*: traverse **left subtree and read visited node content**
- *Post-order*: traverse **all subtrees and read visited node content**

Examples:



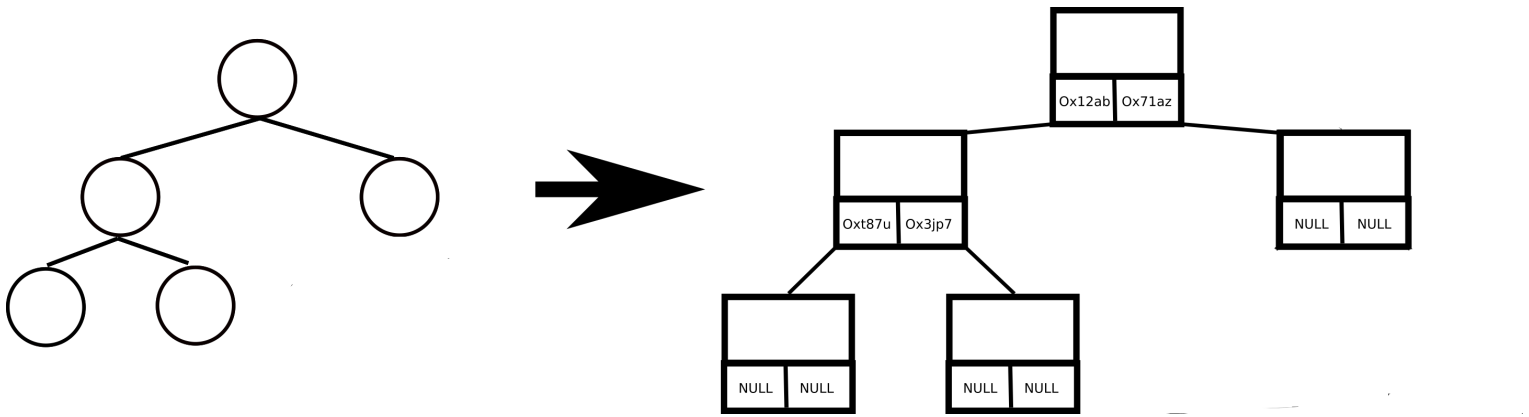
- Left: F, B, A, D, C, E, G, I, H (pre-order)
- Center: A, B, C, D, E, F, G, H, I (in-order)
- Right: A, C, E, D, B, H, I, G, F (post-order)

[3] By default, node contents are read in *pre-order* mode

Binary?

- Binary trees are trees with ***at most*** **two children** per node[2]
- A binary tree is a **special case** of trees[2]

Example of a binary tree (right side: implementation representation):

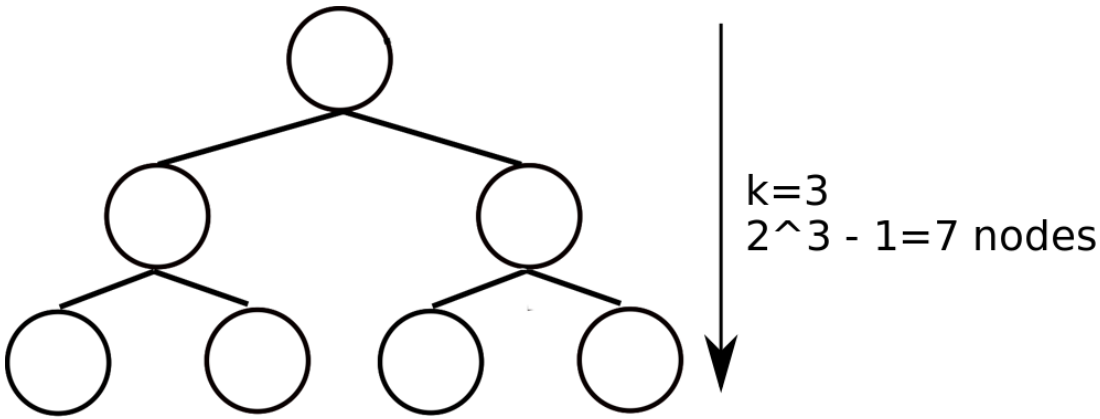


[2]In Data Structure 3, we briefly talked about binary trees for heap sort

A *binary tree* is said *complete* if...

- ... it is a **binary tree** (of course)
- ... it has $2^k - 1$ **nodes** (k being the *binary tree high*)

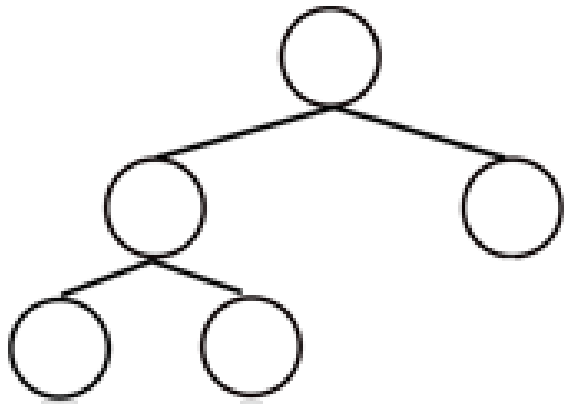
Example:



A binary tree is said *almost complete* if...

- ... it is a **binary tree**
- ... all levels are complete **except the last**
- ... the last level being **partially complete on the *left***

Example:

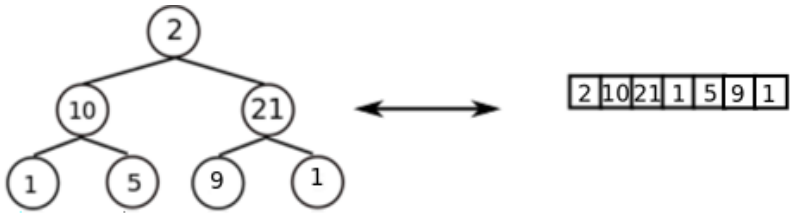


Why this data structure is interesting?

Complete and *almost complete* trees **can be represented as arrays**:

- *left child node value* of node k located at *index* $2 \times k$ in array
- *right child node value* of node k located at *index* $2 \times k + 1$ in the array

Example with a *complete tree*:



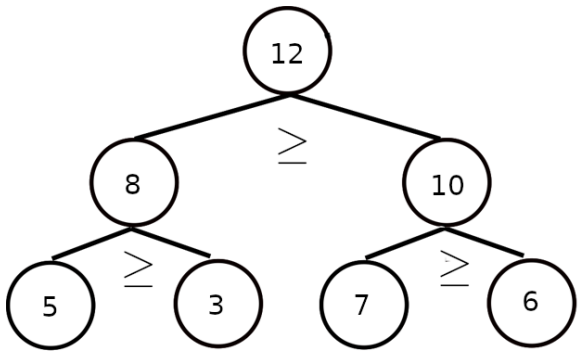
Example with an *almost complete tree*:

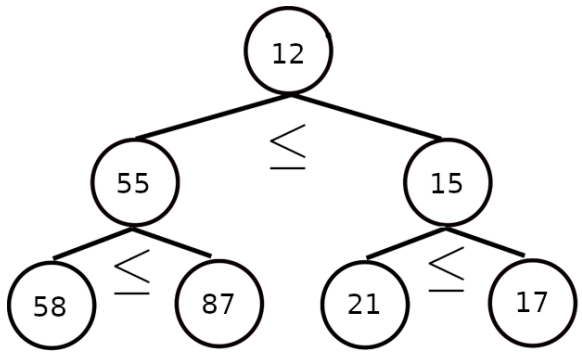
What is a binary heap?

It's a **complete (or almost complete) tree** with one of these properties:

- **node values $\geq$ its children node values** (*max binary heap*)
- or, **node values $\leq$ its children node values** (*min binary heap*)

Examples (left: *max binary heap*, right: *min binary heap*):





Interesting properties with binary heaps:

- Array can be *sorted* using multiple binary heap transformations[4]:
  1. input array is **transformed into a max binary heap**
  2. **root node value is removed from array** (stored in other final array)
  3. a new max binary heap is **built again from the changed array**
  4. **go to (2)**, until there is no more root nodes
- Heap trees can be used as *priority queues*:
  - Root node contains the *highest priority element*
  - Passing to other lower priority elements:
    1. **Removing the root node**
    2. **Rebuilding the heap tree**
    3. New highest priority is **now in the root node, go to (1)**

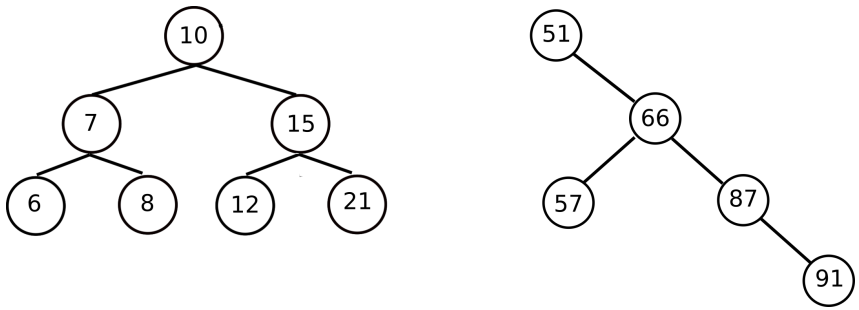
[4] We already saw the heapsort algorithm in course Data Structure 3...

What is a *binary search tree*?

A *binary search tree* (BST) is:

- a ***binary tree***
- with the following properties[5] for each node k:
  - all nodes k' belonging to ***left sub-tree*** of k:
    - **values of** $k' \leq$ **values of** k
  - all nodes k'' belonging to ***right sub-tree*** of k:
    - **values of** $k'' \geq$ **values of** k

Examples of two BSTs:



[5] This also works with values that are not numbers: *there must just be a sorting order*

For searching or inserting in a BST, let's do:

```
Function search_insert(value, address) is
    if address == NULL then
        create_node(value, address);
    else
        if value == address.value then
            return address;
        else
            if value < address.value then
                search_insert(value, address.left)
            else
                search_insert(value, address.right)
            end
        end
    end
end
```

Complexities:

complexities	in average	at worst
searching	$O(log(N))$	$O(N)$
inserting	$O(log(N))$	$O(N)$
deleting	$O(log(N))$	$O(N)$

For searching or inserting in a BST, let's do:

```
Function search_insert(value, address) is
    if address == NULL then
        create_node(value, address);
    else
        if value == address.value then
            return address;
        else
            if value < address.value then
                search_insert(value, address.left)
            else
                search_insert(value, address.right)
            end
        end
    end
end
```

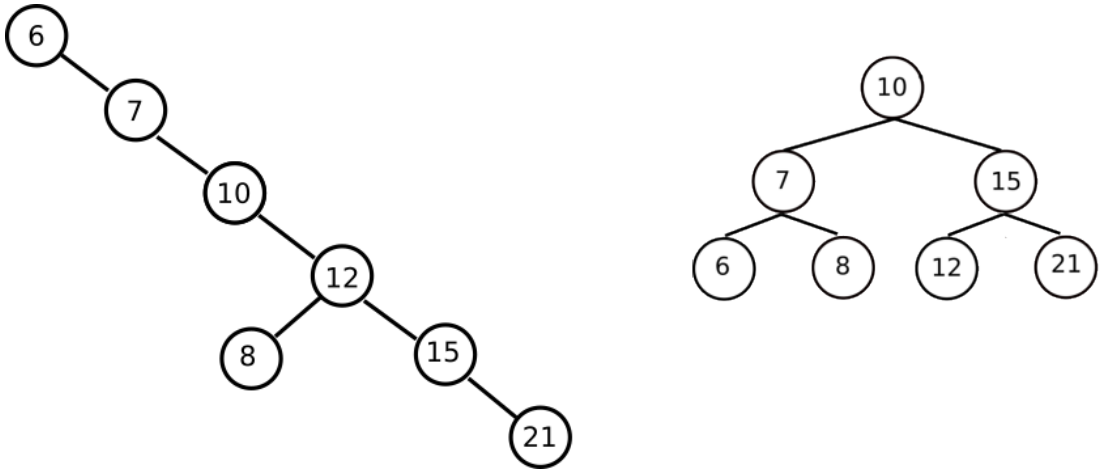
Complexities:

complexities	in average	at worst
searching	$O(log(N))$	$O(N)$
inserting	$O(log(N))$	$O(N)$
deleting	$O(log(N))$	$O(N)$

Note

- *searching*, *inserting* and *deleting* complexities are the same!
- complexities of **balanced BSTs are O(log(N))**
  - because searching (as well as inserting and deleting) is dichotomic
- complexities of **badly balanced BSTs are close to O(N)...**

Example of a *badly balanced* BST (left) and a *well balanced* BST (right):



Both trees have the *same content*, but the *left one is slower to traverse*.

Definition

Each node of a *perfectly balanced* BST verifies:

- number of nodes in left sub-tree = number of nodes in right sub-tree

AVL?

An *AVL tree* is a **self-balancing** *binary search tree*

AVL trees balance themselves by balancing the **high of the sub-trees**:

- do not balance according to the perfectly balanced BST definition
- **easier approach** (*much less* computation)
- good tradeoff

Principle:

1. once a new node is created: its **parent nodes are checked**
2. in case there is an unbalanced parent node we **locally fix it**:
   - addresses of neighbor nodes are **swapped** so highs are balanced
   - addresses swapping is performed thanks to ***rotations***

Rotations?

Rotations are elementary AVL operations to **rebalance the tree**:

- *unbalanced node*: **difference between sub-tree highs ≥ 2**
- sub-trees of *unbalanced nodes* are **rotated**
- rebalanced nodes **keep BST properties**

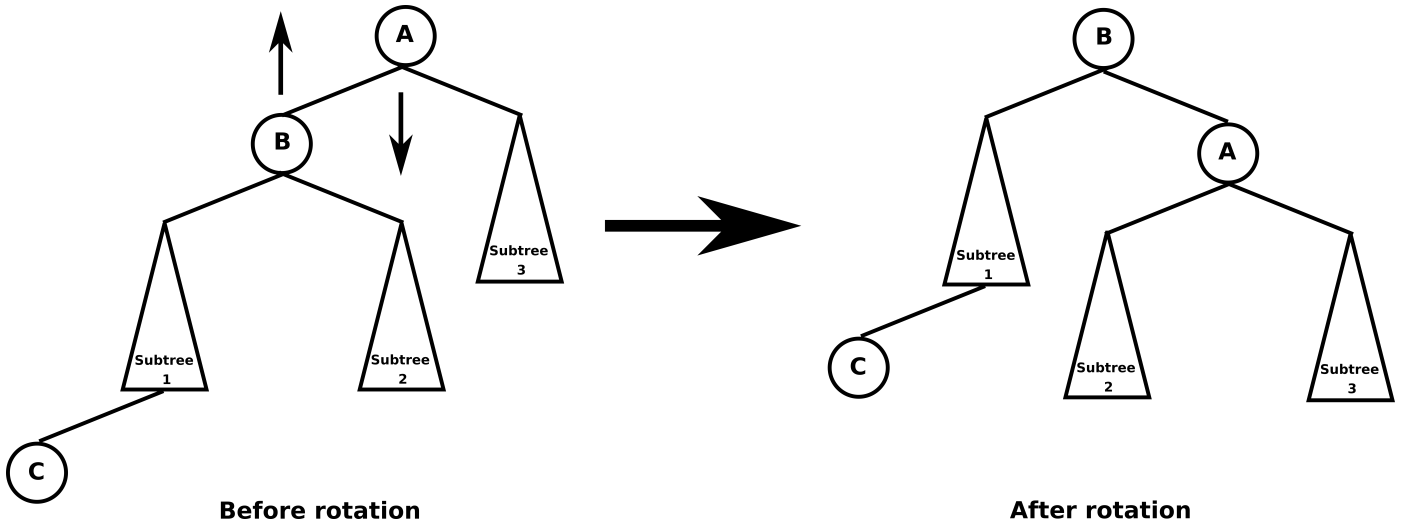
[6] A is the unbalanced node (high difference ≥ 2). C is the newly added node.

Rotations?

Rotations are elementary AVL operations to **rebalance the tree**:

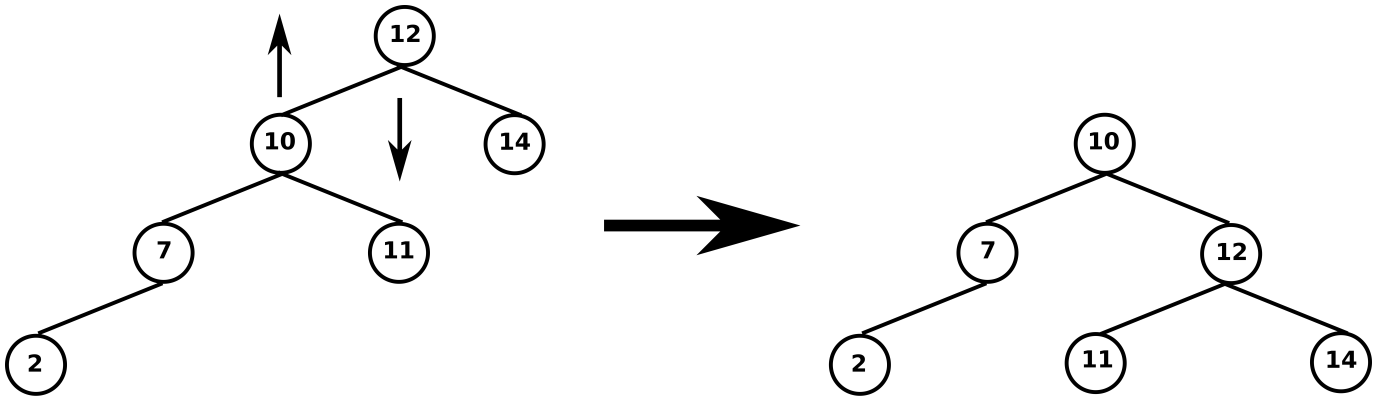
- *unbalanced node*: **difference between sub-tree highs ≥ 2**
- sub-trees of *unbalanced nodes* are **rotated**
- rebalanced nodes **keep BST properties**

Simple rotation[6] (left case):



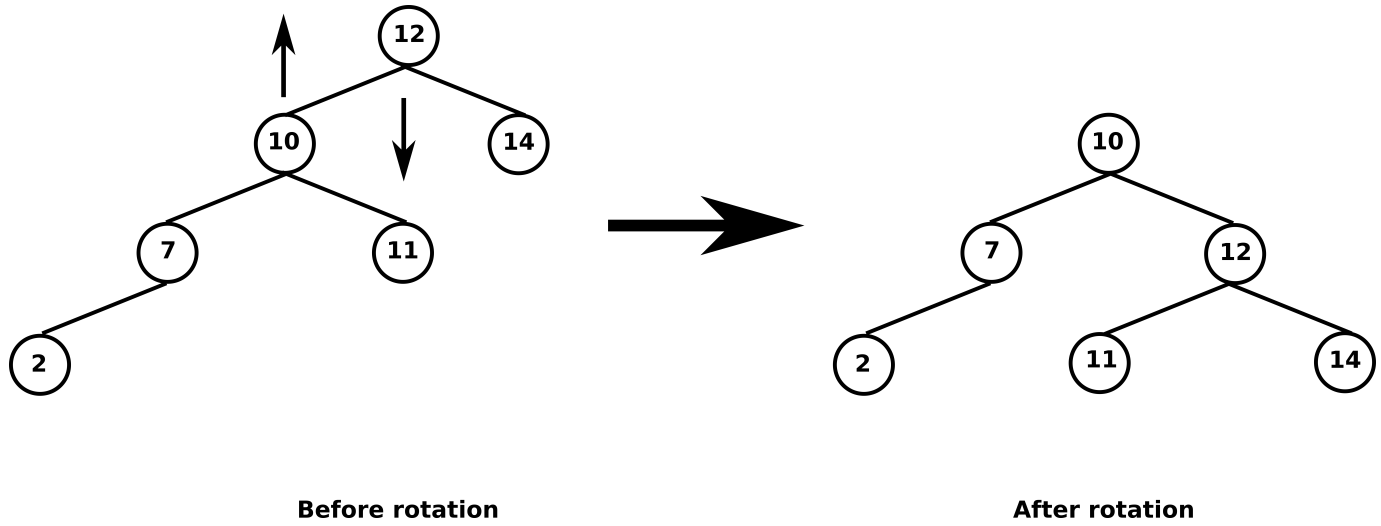
[6] A is the unbalanced node (high difference ≥ 2). C is the newly added node.

Example of *simple rotation* rebalancing:



Before rotation **After rotation**

Example of *simple rotation* rebalancing:

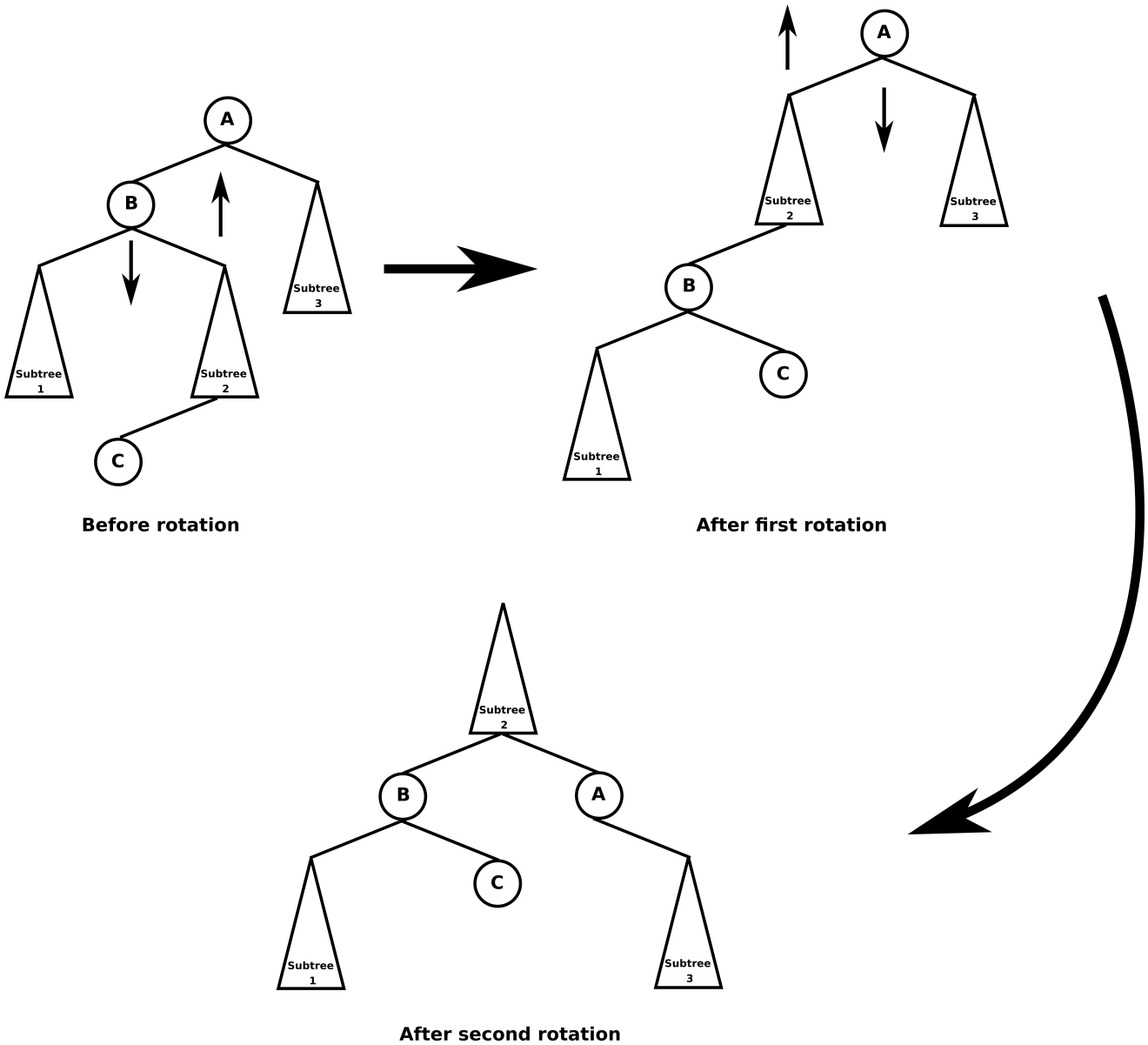


Note

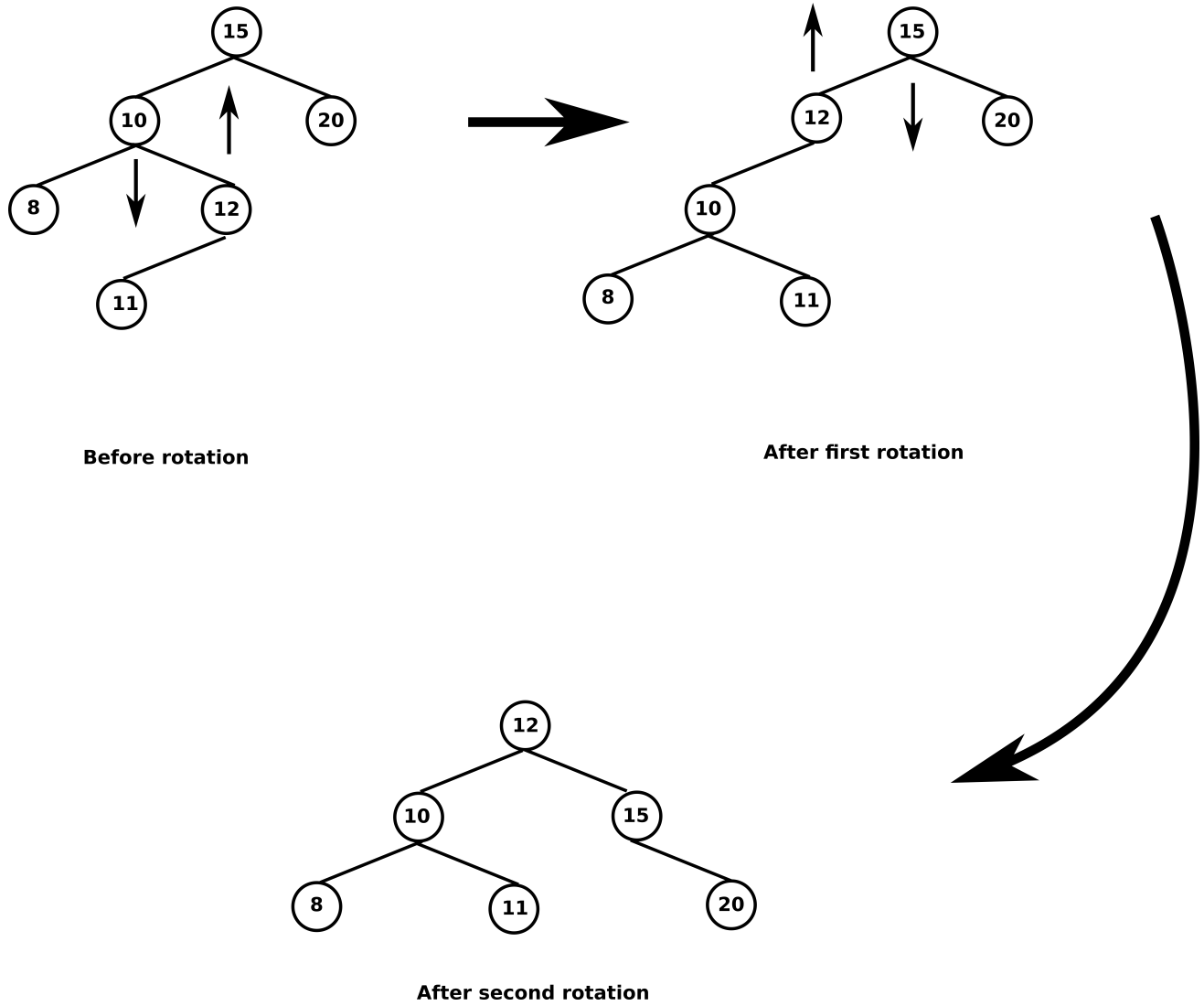
As you can see: *simple rotation* preserves *BST* properties:

- the tree is **still a *binary tree***
- all nodes k' belonging to ***left sub-tree*** of k:
  - **values of** $k' \leq$ **values of** k
- all nodes k'' belonging to ***right sub-tree*** of k:
  - **values of** $k'' \geq$ **values of** k

Double rotation (left case):



Example of *double rotation* rebalancing:



Complexities of a *AVL tree*:

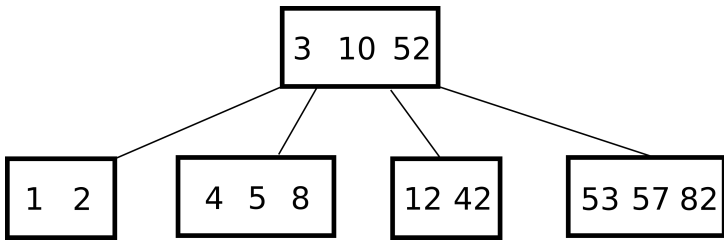
complexities	**in average**	**at worst**
searching	$O(log(N))$	$O(log(N))$
inserting	$O(log(N))$	$O(log(N))$
deleting	$O(log(N))$	$O(log(N))$

B-trees?

B-trees are also **self-balancing trees** *but*:

- unlike *BSTs*, internal nodes **can have more than 2 child nodes**
- each internal node can have **k keys and k+1 addresses**
- **internal nodes are called pages**

Example of a *B-tree*:



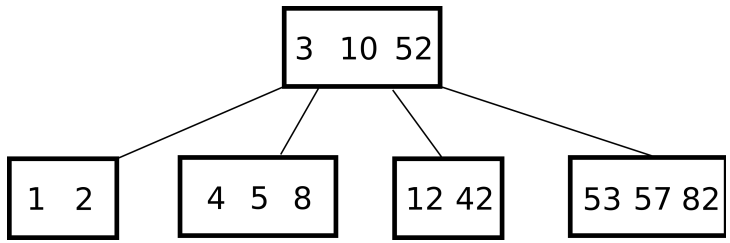
[11] Example of applications using it: *databases* and *file systems*. Here, most of the time, the number of keys cannot fit main memory: but *B-trees* allow to search efficiently in large blocks of data with few memory accesses (= few keys in main memory). Very often: B-tree node size = disk block size.

B-trees?

B-trees are also **self-balancing trees** *but*:

- unlike *BSTs*, internal nodes **can have more than 2 child nodes**
- each internal node can have **k keys and k+1 addresses**
- **internal nodes are called pages**

Example of a *B-tree*:

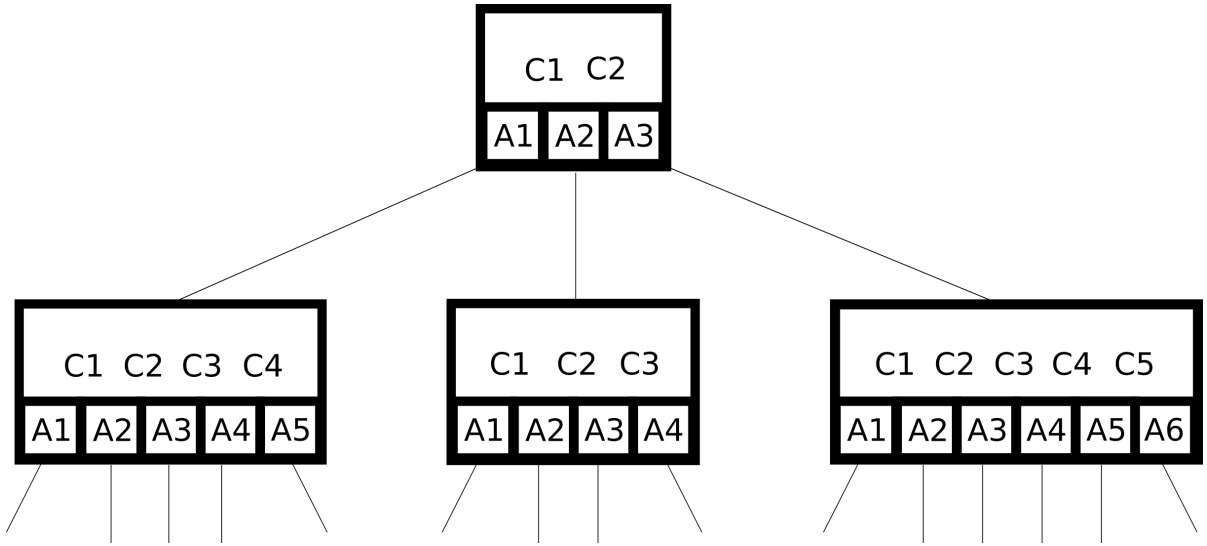


Note

B-trees are interesting for **for reading and writing large blocks of data**[11]

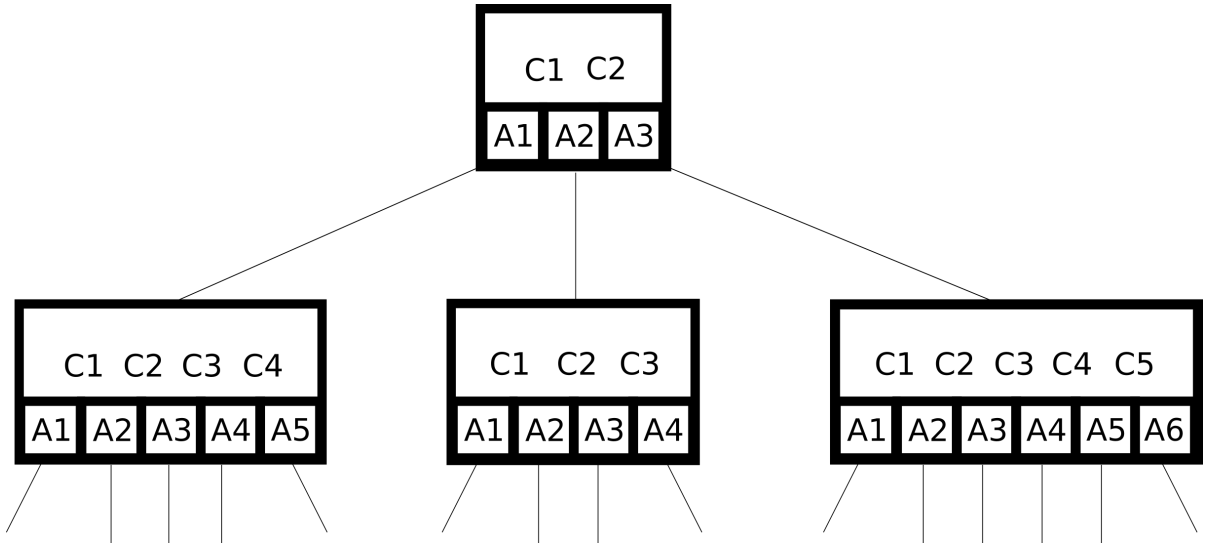
[11] Example of applications using it: *databases* and *file systems*. Here, most of the time, the number of keys cannot fit main memory: but *B-trees* allow to search efficiently in large blocks of data with few memory accesses (= few keys in main memory). Very often: B-tree node size = disk block size.

B-trees are built as follows:



- For each node, **keys are ordered**: C1 < C2 < C3 < C4...
- Splits are organized like a generalized *BST*, example for the first page:
  - keys < key C1 belong to the node pointed by A1
  - keys > key C3 and < C4 belong to the node pointed by A4
  - keys > key C4 belong to the node pointed by A5

B-trees are built as follows:



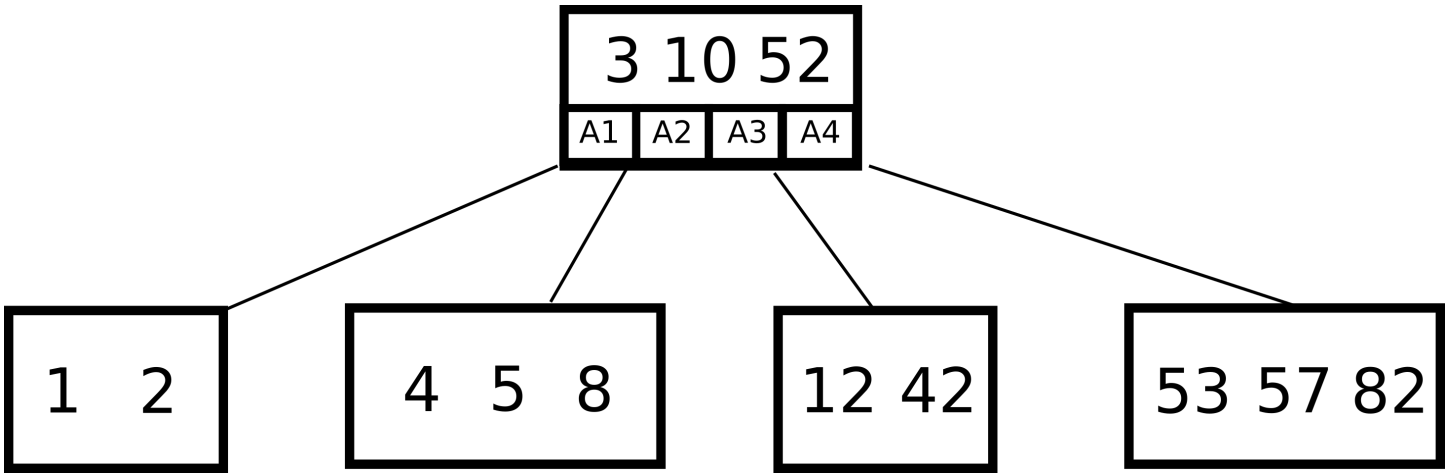
- For each node, **keys are ordered**: C1 < C2 < C3 < C4...
- Splits are organized like a generalized *BST*, example for the first page:
  - keys < key C1 belong to the node pointed by A1
  - keys > key C3 and < C4 belong to the node pointed by A4
  - keys > key C4 belong to the node pointed by A5

B-tree of order N

N minimum number of keys and $2 \times N$ maximum number of keys per page

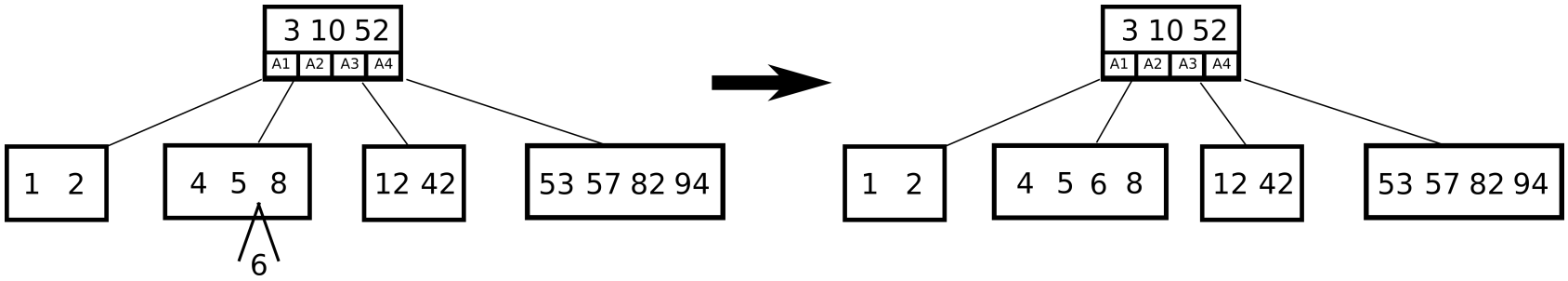
Searching a key in a *B-tree* is quite easy, for example:

- Where is 5? I know that $3 < 5 < 10$: I follow address A2
- Where is 42? I know that $10 < 42 < 52$: I follow address A3
- Where is 57? I know that $57 > 52$: I follow address A4
- ...

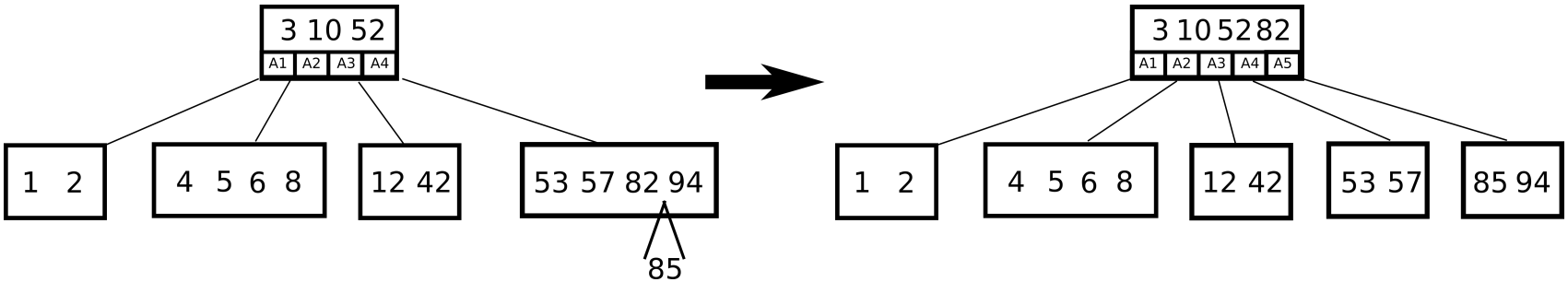


Examples of key insertions in a *B-tree* of order 2:

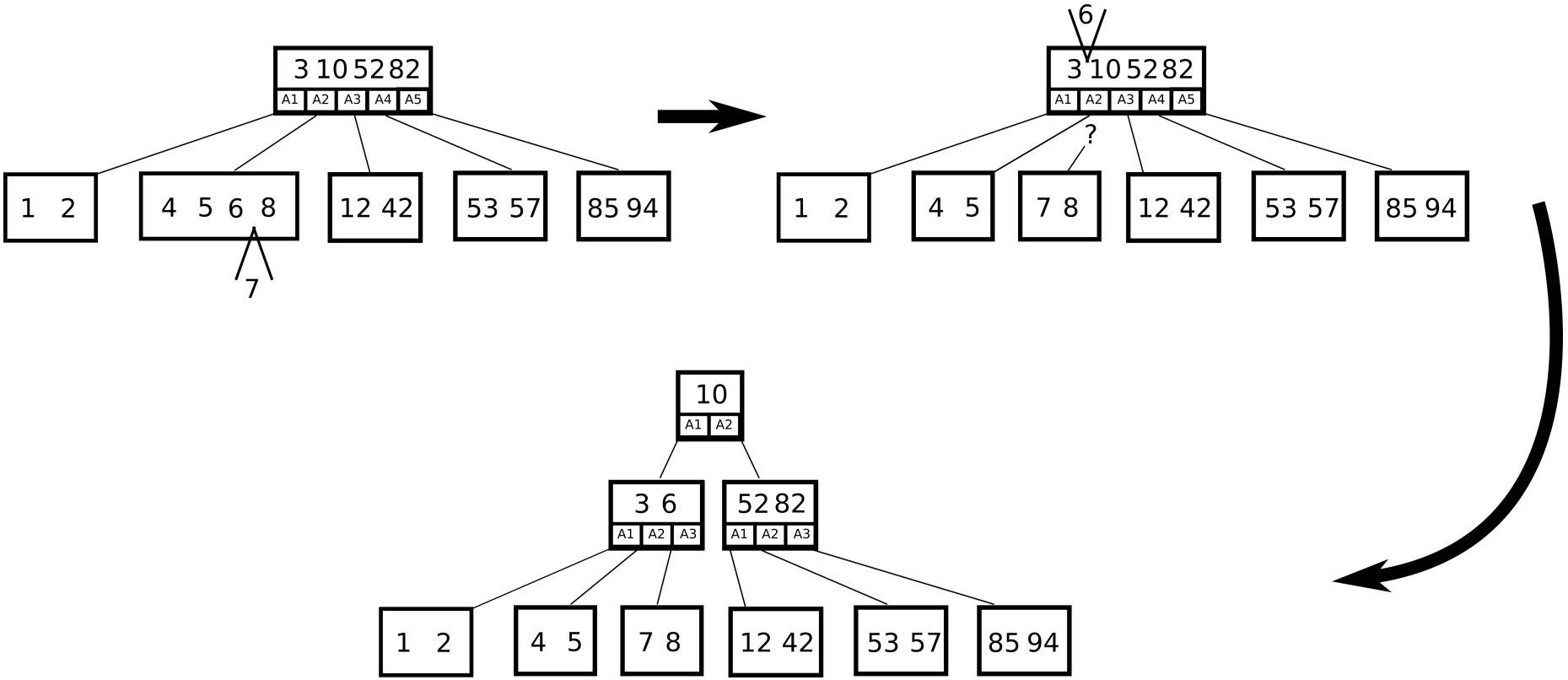
In a *non saturated leaf node*:



In a *saturated leaf node* (B-tree order 2: max number of keys = 4):



In a saturated node with a saturated root node:



Complexities of a *B-tree*:

complexities	**in average**	**at worst**
searching	$O(log(N))$	$O(log(N))$
inserting	$O(log(N))$	$O(log(N))$
deleting	$O(log(N))$	$O(log(N))$

- *Complete and half complete binary trees* $\Leftrightarrow$ arrays
- *Binary heaps*: useful structure for **sorting** and **priority queues**
- *Binary search trees*:
  - if **well-balanced**: **best insertion/deletion/reading complexities**
  - **badly balanced**: complexities **close to those of linked-list**
- *AVL trees*:
  - *BST* with **self-balancing properties**
  - **best insertion/deletion/reading complexities**
- *B-trees*:
  - well suited for large blocks of data (nodes can have > 2 children)
  - sort of **generalized *BST*** (it's not binary! despite the "B-" prefix)
  - **self-balancing**
  - **best insertion/deletion/reading complexities**